



Protecting LLM Prompt Traffic Beyond TLS:  
Evaluating Browser-to-Server Application-Layer Encryption

*A Security Evaluation of  
Traffic Visibility, Integrity, Replay Protection, and Trust*

Aura Gaines  
Cameron Faulkner  
Filip Dziembowski

University of California, Berkeley  
School of Information  
Cyber 210: Network Security

Instructor: Professor Richard Cziva  
27 July 2026

## Table of Contents

|                                   |    |
|-----------------------------------|----|
| 1. Introduction                   | 3  |
| 2. Background and System Overview | 4  |
| 3. Threat Model                   | 6  |
| 4. CIA Triad                      | 7  |
| 5. Security Testing and Controls  | 8  |
| 6. Security Findings              | 9  |
| 7. Security Relevance             | 10 |
| 8. Limitations and Future Work    | 12 |
| 9. Conclusion                     | 13 |
| 10. References                    | 13 |
| APPENDIX A                        | 15 |
| APPENDIX B                        | 16 |

## **1. Introduction**

### **1.1 The Privacy Gap**

A message can be encrypted in transit and still be readable by systems that were never intended to use it.

LLM users may submit source code, incident details, credentials, or personal information through a browser. TLS protects the connection, but once it is terminated, plaintext may become visible to intermediaries before reaching the process that actually needs to read it.

The security problem is therefore not simply whether the traffic is encrypted. It is where the plaintext appears and how many systems can access it.

### **1.2 Research Focus**

Our project examines how browser to server application layer encryption changes what intermediaries can see compared with TLS alone for sensitive LLM style prompt traffic.

To investigate this question, we built a browser based chat prototype that encrypts messages before they leave the browser and keeps them encrypted until they reach the intended server process. We then compared what different observers could see and tested how the encrypted channel responded to active attacks.

### **1.3 Significance**

As organizations use LLMs for sensitive work, every system that can access prompt plaintext expands the security boundary and creates another opportunity for exposure. Application layer encryption may narrow that boundary by limiting decryption to the process that needs the content. That protection is conditional: if an intermediary can alter the browser-delivered encryption code, it can capture plaintext before encryption or substitute a different key. The project therefore evaluates both the benefit of securing the data channel and the residual risk created by an untrusted code-delivery channel.

---

## **2. Background and System Overview**

### **2.1 LLM Prompt Exposure**

Browser based LLM requests often pass through proxies, gateways, filters, and logging systems. Any component that receives decrypted traffic may be able to inspect or store the prompt.

### **2.2 TLS and the Plaintext Boundary**

TLS prevents passive observers from reading application data, but a TLS terminating intermediary can inspect plaintext inside the connection. In our test environment, mitmproxy represented this observer. Application layer encryption moved the plaintext boundary to the intended server process, leaving the proxy able to observe the connection and metadata but not the message content.

### **2.3 Application-Layer Encryption**

Application-layer encryption protects message content inside the TLS connection rather than just the connection itself. Where TLS secures the channel between two endpoints, application-layer encryption ensures that a system terminating that channel still cannot read the payload. We used Hybrid Public Key Encryption (HPKE, RFC 9180), which combines asymmetric key encapsulation with authenticated symmetric encryption in a single well-analyzed primitive. This avoided the implementation risk of assembling those properties by hand and kept nonce and key-schedule management inside the library.

### **2.4 System Architecture (Appendix A)**

The prototype runs three containers on a shared internal Docker network. The mitmproxy container terminates TLS on port 443 and routes traffic by hostname to either the Next.js browser interface or the FastAPI echo server. Neither backend publishes a port to the host, so all browser traffic from the host must pass through mitmproxy. This arrangement allows the proxy to inspect all

WebSocket frames after TLS decryption, which is the condition the experiment tests. A diagram of the full architecture is included in Appendix A.

## 2.5 Cryptographic Design

The suite is DHKEM(X25519, HKDF-SHA256) with HKDF-SHA256 and ChaCha20-Poly1305. The server holds static X25519 and Ed25519 keypairs stored as environment variables. The browser derives its keypairs deterministically from a BIP-39 mnemonic using PBKDF2 followed by domain-separated HKDF expansions, giving it a recoverable identity without server-side key storage. Each connection uses two independent one-directional HPKE contexts, one per direction, so nonce counters advance separately and the library manages them without application involvement. Every message also binds a 37-byte additional authenticated data structure containing an application label, a session identifier derived from the handshake transcripts, a direction token, a frame-type code, and the sequence number, which ties each ciphertext to its exact position in a specific session and direction.

## 2.6 Message Flow

The browser begins by fetching the server's public keys and verifying an Ed25519 signature against a pinned copy of the server identity. It then opens a WebSocket and sends a hello frame containing its own public keys, a KEM encapsulated key, and a signature over a transcript that includes all four public keys. The server verifies the signature, sets up both HPKE contexts, and returns a server\_hello frame with its own encapsulated key and a countersignature. The browser checks that the server\_hello transcript reflects its own keys verbatim, computes a session identifier from both transcripts, and begins sending sealed msg frames. Any verification failure closes the current connection; recovery requires a fresh handshake rather than continuing the failed session.

---

### **3. Threat Model**

#### **3.1 Assets**

The main asset is the sensitive text entered into the browser. The server response is also protected because it contains the same message on the return path. Both directions require confidentiality and integrity.

#### **3.2 Adversary Model**

The threat model includes passive observers, TLS terminating intermediaries, and active network attackers. A passive observer can capture packets and metadata but cannot break TLS. A TLS terminating intermediary can decrypt the connection and inspect WebSocket traffic. An active attacker may replace public keys, alter ciphertext, replay or reorder messages, reflect ciphertext into another direction or session, or send malformed frames. The attacker is not assumed to control the browser or intended server.

#### **3.3 Trust Assumptions**

The browser must receive honest application code and the correct pinned server identity. The cryptographic libraries must work as intended, and the server private key must remain secret. The intended server process is trusted to read the message after decryption. The project protects plaintext from intermediaries, not from the process that must use it.

#### **3.4 Security Goals**

The primary goal is to prevent intermediaries from reading prompt contents after TLS termination. The system should also detect key substitution, ciphertext modification, replay, reordering, reflection, and invalid protocol states.

When a security check fails, the connection should close without returning plaintext or continuing in an uncertain state.

#### **3.5 Out-of-Scope**

The primary cryptographic threat model does not claim protection against a compromised browser, compromise of the intended server or its private keys, privileged access to server memory, or denial of service. We separately tested modification of browser-delivered code—not as an attack on HPKE itself, but to demonstrate the boundary of any design whose encryption code arrives through an attacker-controlled channel. User authentication, production key management, and a complete secure-software-delivery design remain outside the scope of this prototype.

---

## **4. CIA Triad**

### **4.1 Confidentiality**

Confidentiality is the primary goal. TLS protects against passive observation, while application layer encryption keeps the message unreadable at TLS terminating intermediaries. Metadata and plaintext at the intended server remain outside this protection.

### **4.2 Integrity**

Authenticated encryption, server identity verification, signed handshake data, and sequence numbers protect message integrity. Testing confirmed that modified, replayed, reordered, and reflected ciphertext was rejected.

### **4.3 Availability**

Availability is secondary. Invalid messages close the connection, and the prototype does not prevent traffic blocking, flooding, or forced reconnects.

### **4.4 Tradeoffs**

Closing the connection after suspicious input protects confidentiality and integrity but makes disruption easier. The system stops rather than continuing in an uncertain state.

---

## **5. Security Testing and Controls**

### **5.1 Observer Comparison**

We compared what a passive network observer and a TLS terminating intermediary could see. Wireshark showed encrypted TLS traffic in both modes. Mitmproxy could read prompts with TLS alone, but saw only ciphertext when application layer encryption was enabled.

### **5.2 Identity and Key Substitution**

Testing attempted to replace the pinned server identity, server encryption key, signatures, and handshake values. The client rejected the modified values before sending protected messages.

### **5.3 Handshake and Protocol State**

We sent messages before the handshake completed, repeated handshake messages, altered required fields, and introduced unexpected frame types. The system rejected each invalid state and closed the connection.

### **5.4 Ciphertext Tampering**

Ciphertext was modified, replaced, and reused to test authenticated encryption. Each change caused authentication to fail, and no altered plaintext was accepted or echoed.

### **5.5 Replay, Ordering, and Reflection**

Testing included duplicate sequence numbers, sequence gaps, older messages, messages sent out of order, and ciphertext replayed across sessions. We also reflected ciphertext between the client to server and server to client directions. The system rejected each attempt without producing duplicate echoes or plaintext.

### **5.6 Malformed Input Handling**

Malformed tests included missing fields, invalid sequence values, incorrect ciphertext encodings, unknown frame types, and invalid JSON. The system rejected these inputs without falling back to plaintext processing.



## 5.7 Plaintext and Logging Checks

Unique test messages were searched across client, server, container, and browser output. No unexpected plaintext was found in the tested logs or browser console. Prompt contents remained visible only where expected, including the TLS terminating proxy in TLS only mode and the intended server after decryption.

## 5.8 Defensive Controls

Server identity pinning, signed handshake data, authenticated encryption, sequence tracking, and session and direction binding protected the channel. Invalid messages closed the connection, while generic client errors limited unnecessary information exposure.

---

# 6. Security Findings

## 6.1 Successful Controls

The core security claim held. TLS terminating intermediaries could read prompts protected by TLS alone but saw ciphertext when application layer encryption was enabled. TThe tested active attacks against the HPKE data channel were rejected without altered plaintext, duplicate echoes, or unexpected log exposure.

## 6.2 Redteam Findings

Red team testing found four implementation issues involving canonicalization, error consistency, and information disclosure. The client accepted a noncanonical Base64URL form of the correct server pin, malformed sequence and ciphertext values produced inconsistent responses, and some malformed inputs exposed Python specific exception text.

None of these issues bypassed encryption or server identity verification. Before final evaluation, the implementation added canonical encoding checks, standardized malformed input handling, and replaced internal exceptions with generic client facing errors.

### **6.3 Accepted and Out-of-Scope Risks**

Accepted risks include plaintext at the intended server, endpoint compromise, observable metadata, and denial of service. These fall outside the narrower goal of protecting prompt contents from intermediaries.

### **6.4 Code Delivery Risk**

A TLS terminating proxy that modifies browser delivered JavaScript can capture the prompt before encryption or alter the encryption key. This attack falls outside the HPKE data-channel guarantee but was tested to expose the system's larger trust boundary: protecting the data channel is insufficient when an attacker controls the code that creates the ciphertext.

### **6.5 Overall Assessment**

The prototype met its primary goal by protecting prompt contents from a TLS terminating intermediary and rejecting the tested active attacks. Its protection still depends on trusted endpoints, secure key handling, and trusted client code.

---

## **7. Security Relevance**

### **7.1 LLM Prompt Privacy**

Browser-based LLM interfaces regularly handle sensitive content including source code, legal questions, medical details, and internal documents. The typical request path passes through proxies, gateways, and logging middleware, any of which can read the prompt if TLS terminates there. Application-layer encryption moves the confidentiality boundary to the intended application endpoint, so intermediaries on the data channel see only ciphertext after TLS termination, assuming the browser code and pinned server identity remain trusted. For organizations where compliance or data access requirements limit which systems may process sensitive content, this distinction has practical value beyond what TLS alone provides.

## 7.2 Security Beyond TLS

TLS protects the channel between two negotiated endpoints but does not constrain what those endpoints do with the data after decryption. A system that is authorized to terminate TLS, such as a corporate proxy or cloud load balancer, sees the same plaintext as the intended destination.

Application-layer encryption adds a property TLS does not provide: decryption is limited to the holder of the private key associated with the pinned server identity, regardless of the network path. The two layers address different parts of the threat model and are complementary rather than redundant.

## 7.3 Browser Security Implications

The browser is the origin of the encryption, which means its trustworthiness determines whether the protection holds. If the JavaScript performing encryption is replaced before execution, an attacker can capture the plaintext prompt or substitute a different key without the user being aware. Both the code-delivery channel and data channel pass through the same TLS-terminating proxy, which can rewrite the delivered JavaScript. The prototype documents this as an accepted limitation; addressing it requires removing the server from the code delivery trust path, described in section 8.4.

## 7.4 Threat Modeling Lessons

Threat modeling must distinguish attacker position from attacker capability. Passive observers, TLS terminating intermediaries, and attackers who control client code occupy different trust boundaries and require different controls. Keeping those boundaries separate prevented the project from overstating what the encrypted channel could protect.

## 7.5 Broader Applicability

The problem this project addresses is not specific to LLMs. Any browser application that sends sensitive content through infrastructure it does not fully control faces the same gap between TLS protection and plaintext access at intermediaries. Healthcare, legal, financial, and enterprise tools operating behind centralized inspection gateways all share this structure. Application-layer encryption

narrows the set of systems that must be trusted with plaintext, which reduces the scope of what needs to be protected and audited without replacing other controls.

---

## **8. Limitations and Future Work**

### **8.1 Prototype Limitations**

The server is an echo process rather than a real LLM, and the deployment runs on a private test network with a self-generated certificate authority. The system has no user authentication, rate limiting, persistent user-session management, or key rotation. Browser identity depends on a BIP-39 mnemonic with no passphrase, making the mnemonic the sole secret for the browser side. These constraints were sufficient for the research question but limit direct application of the results to production environments.

### **8.2 Threat Model Limitations**

The threat model assumes honest browser code, secure server keys, and correct cryptographic libraries. The controlled mitmproxy environment was sufficient for the research question but does not represent every production deployment, cloud configuration, or privileged attacker.

### **8.3 Client Code Delivery**

Future work should address how browser code is delivered and verified, not only how messages are encrypted after execution begins.

### **8.4 Packaged or Pinned Clients**

Removing the application server from the code-delivery trust path is the direct mitigation for the risk described in Sections 6.4 and 7.3. A browser extension installed from a vetted store, a signed desktop application, or a mobile application distributed through an authenticated platform channel can deliver client code through a path that the TLS-terminating proxy cannot modify. Because the

encryption code is not fetched through the proxied application connection, the proxy cannot rewrite it to capture plaintext before encryption.

Subresource Integrity alone does not solve this threat when the same intermediary can rewrite the HTML containing the integrity value. It helps only when the hash or loading document is independently trusted. A production design therefore needs a code-authenticity root outside the proxied application path, such as signed extensions or application packages with authenticated updates.

## **8.5 Hardening and Future Testing**

A production version would need stronger key management, authentication, rate limiting, monitoring, and deployment controls. Additional testing should include browser extensions, dependency compromise, memory exposure, larger traffic volumes, and real cloud infrastructure.

---

## **9. Conclusion**

HTTPS can secure a connection without securing every system behind it. This project made that hidden boundary visible: a TLS terminating intermediary could read prompts protected by TLS alone but lost access when encryption began in the browser.

The encrypted channel also resisted key substitution, tampering, replay, reordering, reflection, and malformed input. Client code tampering exposed the broader lesson that network security depends not only on strong encryption, but also on where plaintext appears, where trust begins, and who controls the code that creates the ciphertext.

---

## **10. References**

Barnes, R., Bhargavan, K., Lipp, B., and Wood, C. A. *Hybrid Public Key Encryption*. RFC 9180, February 2022.

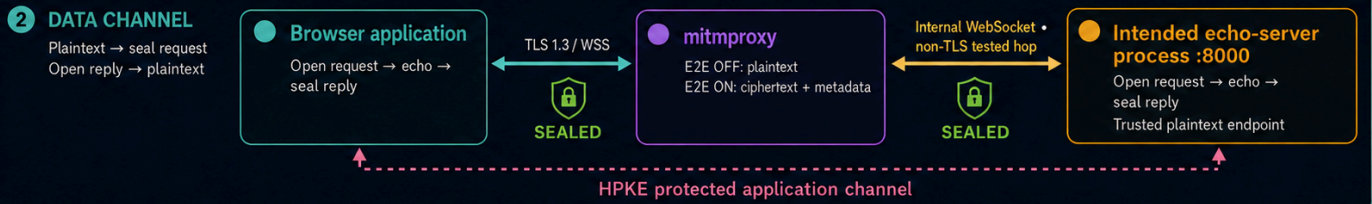
Gaines, A., Faulkner, C., & Dziembowski, F. (2026). *Protecting LLM prompt traffic beyond TLS: Evaluating browser-to-server application-layer encryption*. GitHub.  
<https://github.com/Goat20726/cyber210-Browser-to-Server-e2e>

## APPENDIX A

### EchoVault Architecture

#### EchoVault : Securing Data Channel

**RESEARCH QUESTION** How does application-layer HPKE change with a TLS-terminating intermediary, and is it more secure compared with TLS alone?



**Protecting the data channel is not enough when the adversary can control the code that creates the ciphertext.**

HPKE protects the message after TLS termination and across untrusted hops, but if the client runs malicious code, the plaintext exists before sealing and can be exfiltrated.

#### TRUST RESTORATION

Use a locally owned or independently pinned client to remove mutable code delivery from the intermediary.

HPKE base mode • Pinned Ed25519 server authentication • X25519 • HKDF-SHA256 • ChaCha20-Poly1305

## APPENDIX B

| Cyber 210 Final Project Redteam Evidence Tracker                                                         |      |                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Session 1: Core three-model and observer comparison                                                      |      |                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                          |
| Capture true plaintext/no-TLS traffic on the wire                                                        | PASS | tcpdump capture on port 8000 showed the exact test token readable in plaintext                                                                                                                                                               | The token AURA-PLAINTEXT-NOTLS-2026-002 was visible in cleartext on the internal non-TLS hop between mitmproxy and the echo server.                                                                                                                                                                                                                                                      |
| Capture TLS-only traffic from a passive-observer perspective with Wireshark                              | PASS | Wireshark capture of localhost port 443 TLS traffic; packet-byte search showed the unique plaintext token was not present.                                                                                                                   | A passive observer could see TLS traffic and metadata, but not the plaintext token.                                                                                                                                                                                                                                                                                                      |
| Confirm plaintext visibility at the TLS-terminating proxy with E2E off                                   | PASS | mitmweb WebSocket capture showing the complete fake SSN/password prompt readable after TLS termination                                                                                                                                       | Plaintext was visible. IMPLEMENTATION BUG: Server closed with code 4001 due to a malformed seq parsing error.                                                                                                                                                                                                                                                                            |
| Capture a successful TLS-only plaintext echo round trip using the intended plaintext path                | PASS | mitmweb shows the full plaintext request and echo response on /ws/plain; browser DevTools shows the readable echoed token returned to the client                                                                                             | Successful TLS-only round trip. The TLS-terminating proxy could read both directions of the application-layer plaintext.                                                                                                                                                                                                                                                                 |
| Capture TLS + HPKE traffic from a passive-observer perspective with Wireshark                            | PASS | Wireshark capture of localhost port 443 traffic; packet-byte search returned "No packet contained" for AURA-WIRESHARK-E2E-2026-001                                                                                                           | Passive observer could see TLS traffic and metadata, but not the unique plaintext token.                                                                                                                                                                                                                                                                                                 |
| Confirm ciphertext visibility and prompt confidentiality at the TLS-terminating proxy with E2E on        | PASS | mitmweb WebSocket capture showing encrypted ct content while the fake bank account and PRINCESS-DONUT-2026 recovery code are not readable                                                                                                    | mitmproxy could observe the WebSocket traffic but could not read the prompt.                                                                                                                                                                                                                                                                                                             |
| Capture the full encrypted round trip: ciphertext at mitmproxy and readable echo at the intended browser | PASS | Browser shows readable echoed token while mitmweb shows only encrypted ct content and does not reveal token                                                                                                                                  | Intended browser received the readable echo successfully; the TLS-terminating proxy observed the traffic but could not read the token.                                                                                                                                                                                                                                                   |
| Create one observer-comparison summary covering plaintext, TLS-only, and TLS + HPKE                      | DONE | Markdown comparison summarizing what each observer could see across the three protection models                                                                                                                                              | Plaintext was readable without TLS. TLS protected content from passive observation but exposed plaintext at TLS termination. TLS + HPKE kept prompt content hidden from the TLS-terminating proxy while the intended browser still received the readable echo.                                                                                                                           |
| Session 2: Identity, key-substitution, and handshake attacks                                             |      |                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                          |
| Wrong server pin                                                                                         | PASS | Browser capture showing a non-canonical base64url variant of the pinned Ed25519 key was accepted as verified, followed by a second browser capture showing a genuine byte-level key mismatch was rejected and no E2E channel was established | Genuine key substitution was rejected as intended. SECURITY-RELEVANT VALIDATION WEAKNESS: Changing only the final base64url character from k to m produced a visibly different pin string that still verified because both strings decoded to the same 32-byte Ed25519 key. The browser therefore accepts visually different non-canonical encodings of the same cryptographic identity. |
| Modify the server Ed25519 identity key returned by /pubkey                                               | PASS | Browser capture showing the substituted server identity key was rejected and no E2E channel was established                                                                                                                                  | The proxy modified the server_ed25519 value returned by /pubkey. The browser rejected the substituted identity key and failed closed, preventing channel establishment.                                                                                                                                                                                                                  |
| Modify the server X25519 encryption key returned by /pubkey                                              | PASS | Browser capture showing the substituted server encryption key was rejected and no E2E channel was established                                                                                                                                | The proxy modified the server_x25519 value returned by /pubkey. The browser rejected the tampered encryption key because the original signature no longer matched the modified key material and failed closed, preventing channel establishment.                                                                                                                                         |



|                                                                                             |      |                                                                                                                                          |                                                                                                                                                                                                                                     |
|---------------------------------------------------------------------------------------------|------|------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Modify the /pubkey signature                                                                | PASS | Browser capture showing the tampered /pubkey signature was rejected and no E2E channel was established                                   | The proxy modified the sig value returned by /pubkey while leaving the server keys unchanged. The browser rejected the invalid signature and failed closed, preventing channel establishment.                                       |
| Tamper with server_hello handshake material                                                 | PASS | Browser capture showing the tampered server_hello handshake caused the encrypted channel to be torn down and prevented E2E establishment | The proxy modified the server_hello enc value in transit while leaving the original signature unchanged. The browser detected the invalid handshake material and failed closed by tearing down the encrypted channel.               |
| Send msg before hello                                                                       | PASS | Terminal capture showing a fresh WebSocket connection sent a msg frame before hello and the server closed the connection with code 4001  | The server rejected the out-of-order msg frame while in AWAIT_HELLO state and closed the WebSocket with protocol error code 4001. No session was established and no plaintext fallback occurred.                                    |
| Test malformed hello fields: missing/invalid enc, sig, or browser public keys               | PASS | Terminal capture showing multiple malformed hello variants were rejected with WebSocket close code 4001                                  | The server rejected hello frames with missing enc, missing sig, missing browser public keys, and invalid cryptographic values. Each malformed handshake failed closed with protocol error code 4001 and no session was established. |
| Send a second hello after the session is already established                                | PASS | Terminal capture showing a valid first hello established the session, then a second hello was rejected with WebSocket close code 4001    | The server accepted the initial valid hello and returned server_hello, then rejected a second hello in the ESTABLISHED state and closed the connection with protocol error code 4001.                                               |
| Send an unknown or incorrect frame type for the current protocol state                      | PASS | Terminal capture showing an unknown frame type was rejected with WebSocket close code 4001                                               | The server rejected the bogus_frame type while in AWAIT_HELLO state and closed the connection with protocol error code 4001. No session was established.                                                                            |
| Session 3: Ciphertext tampering and integrity attacks                                       |      |                                                                                                                                          |                                                                                                                                                                                                                                     |
| Change one character in a valid ciphertext                                                  | PASS | mitmproxy logs showing one-character ciphertext modification and connection termination                                                  | Changing one character in a valid ciphertext caused the message to fail without producing a valid plaintext echo.                                                                                                                   |
| Modify seq without modifying the corresponding ct                                           | PASS | mitmproxy logs showing sequence number modification with ciphertext unchanged and connection termination                                 | Changing the sequence number without modifying the corresponding ciphertext caused the message to fail without producing a valid plaintext echo.                                                                                    |
| Replace one message's ciphertext with another message's ciphertext                          | PASS | mitmproxy logs showing the first message ciphertext saved, substituted into the second message, and connection termination               | Replacing one message's ciphertext with ciphertext from another message caused the message to fail without producing a valid plaintext echo.                                                                                        |
| Confirm tampered messages produce no valid plaintext or valid echo                          | PASS | Browser behavior and mitmproxy logs from ciphertext modification, sequence tampering, and ciphertext substitution tests                  | Across all ciphertext tampering tests, modified messages produced no valid plaintext and no valid echo.                                                                                                                             |
| Confirm tampering failures do not crash the server or expose sensitive internal information | PASS | Server logs showing generic tamper-failure messages and successful post-tamper encrypted echo                                            | Tampering failures did not crash the server or expose plaintext, key material, stack traces, or other sensitive internal information; the server remained available and successfully echoed a subsequent valid encrypted message.   |
| Session 4: Replay, ordering, and reflection attacks                                         |      |                                                                                                                                          |                                                                                                                                                                                                                                     |
| Replay an exact valid encrypted message                                                     | PASS | mitmproxy logs showing an exact valid encrypted message saved, replayed, and followed by connection termination without a duplicate echo | Replaying an exact valid encrypted message caused the connection to terminate without producing a duplicate plaintext echo.                                                                                                         |

|                                                                                                  |      |                                                                                                                                                                                         |                                                                                                                                                                                           |
|--------------------------------------------------------------------------------------------------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Send the same sequence number twice                                                              | PASS | mitmproxy logs showing a second encrypted message changed to reuse the first sequence number and connection termination without a valid echo                                            | Sending the same sequence number twice caused the second message to be rejected without producing a valid plaintext echo.                                                                 |
| Send a sequence gap                                                                              | PASS | mitmproxy logs showing a sequence number changed to create a gap and connection termination without a valid echo                                                                        | Sending a sequence gap caused the out-of-sequence message to be rejected without producing a valid plaintext echo.                                                                        |
| Send an old sequence number after a newer one                                                    | PASS | mitmproxy logs showing an older valid encrypted message injected after a newer valid message and connection termination without a duplicate echo                                        | Sending an old valid encrypted message after a newer one caused the old message to be rejected without producing a duplicate plaintext echo.                                              |
| Send encrypted messages out of order                                                             | PASS | mitmproxy logs showing the second encrypted message injected before the first and connection termination without a valid echo                                                           | Sending valid encrypted messages out of order caused the reordered traffic to be rejected without producing a valid plaintext echo.                                                       |
| Replay ciphertext from an earlier session into a new session                                     | PASS | mitmproxy logs showing ciphertext saved from an earlier encrypted session, injected into a new session at the expected sequence number, and connection termination without a valid echo | Replaying ciphertext from an earlier encrypted session into a new session caused the message to be rejected without producing a valid plaintext echo.                                     |
| Try using client-to-server ciphertext in the server-to-client direction                          | PASS | mitmproxy logs showing client-to-server ciphertext substituted into the server-to-client direction and connection termination without a valid echo                                      | Using client-to-server ciphertext in the server-to-client direction caused the reflected ciphertext to be rejected without producing a valid plaintext echo.                              |
| Try using server-to-client ciphertext in the client-to-server direction                          | PASS | mitmproxy logs showing server-to-client ciphertext substituted into the client-to-server direction and connection termination without a valid echo                                      | Using server-to-client ciphertext in the client-to-server direction caused the reflected ciphertext to be rejected without producing a valid plaintext echo.                              |
| Confirm rejected replay/ordering/reflection attacks do not produce duplicate echoes or plaintext | PASS | Browser behavior and mitmproxy logs from replay, ordering, cross-session replay, and reflection tests                                                                                   | Across all replay, ordering, cross-session replay, and reflection tests, rejected attacks produced no duplicate plaintext echoes and no valid plaintext output from the rejected traffic. |

#### Session 5: Malformed-message and protocol robustness testing

|                                                                      |      |                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                        |
|----------------------------------------------------------------------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Send a msg missing ct                                                | PASS | Standalone malformed-frame harness completed a valid handshake, sent a msg frame without the ct field, and captured the server rejection.                                        | Server rejected the malformed frame with close code 4001 and reason malformed msg frame: 'ct'. No valid echo or plaintext was returned.                                                                                                                                                                                |
| Send a msg missing seq                                               | PASS | Standalone malformed-frame harness completed a valid handshake, sent a msg frame without the seq field, and captured the server rejection.                                       | Server rejected the malformed frame with close code 4001 and reason malformed msg frame: 'seq'. No valid echo or plaintext was returned.                                                                                                                                                                               |
| Test malformed seq values: numeric, empty, invalid hex, wrong length | PASS | Standalone malformed-frame harness completed a valid handshake for each case and tested numeric, empty, invalid-hex, and wrong-length seq values on fresh WebSocket connections. | Malformed seq values were rejected as intended. IMPLEMENTATION BUG: Numeric, empty, and invalid-hex sequence values returned explicit malformed-frame errors, while a wrong-length sequence value returned the generic close reason internal error, showing inconsistent error handling for malformed sequence length. |
| Send a frame missing type                                            | PASS | Standalone malformed-frame harness completed a valid handshake, sent a frame without the type field, and captured the server rejection.                                          | Server rejected the frame as intended with close code 4001 and reason expected msg in ESTABLISHED, got None. No valid echo or plaintext was returned.                                                                                                                                                                  |

|                                                                                        |      |                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                           |
|----------------------------------------------------------------------------------------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Send an unknown type                                                                   | PASS | Standalone malformed-frame harness completed a valid handshake, sent a frame with an unrecognized type value, and captured the server rejection.                                             | Server rejected the unknown frame type as intended with close code 4001 and reason expected msg in ESTABLISHED, got 'not_a_real_type'. No valid echo or plaintext was returned.                                                                                                                                                                           |
| Test malformed ciphertext encoding: empty, truncated, invalid Base64URL                | PASS | Standalone malformed-frame harness completed a valid handshake for each case and tested empty, truncated, and invalid-Base64URL ciphertext values on fresh WebSocket connections.            | Malformed ciphertext encodings were rejected as intended. IMPLEMENTATION BUG: The truncated ciphertext returned an explicit malformed-frame Base64 decoding error, while empty and invalid-Base64URL ciphertext values returned the generic close reason internal error, showing inconsistent error handling for malformed ciphertext input.              |
| Send malformed/non-JSON input                                                          | PASS | Standalone malformed-frame harness completed a valid handshake for each case, sent both non-JSON text and malformed JSON on fresh WebSocket connections, and captured the server rejections. | Server rejected both malformed input cases as intended with close code 4001 and explicit malformed/non-JSON frame reasons. No valid echo or plaintext was returned.                                                                                                                                                                                       |
| Reproduce and classify the previously observed 4001 malformed seq issue                | PASS | Standalone malformed-frame harness completed a valid handshake, resent a wrong-length seq value on a fresh WebSocket connection, and reproduced the same server response observed earlier.   | The malformed sequence was rejected as intended. IMPLEMENTATION BUG: A wrong-length seq value consistently produced close code 4001 with the generic reason internal error rather than a specific malformed-frame validation message, reproducing the previously observed malformed-sequence issue.                                                       |
| Session 6: Plaintext leakage and logging checks                                        |      |                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                           |
| Send a unique token with E2E off and search mitmproxy, server, client, and Docker logs | PASS | mitmproxy WebSocket capture showing the unique E2E-off token in plaintext and Docker log search showing no matching token in mitmproxy, client, or server container logs                     | With E2E disabled, the unique token was visible in plaintext to the intercepting proxy as expected, but was not found in mitmproxy, client, or server Docker logs. No unexpected plaintext logging was observed.                                                                                                                                          |
| Send a different unique token with E2E on and search the same locations                | PASS | mitmproxy WebSocket capture showing ciphertext-only encrypted message frames and Docker log search showing no matching plaintext token in mitmproxy, client, or server container logs        | With E2E enabled, the unique token was absent from mitmproxy WebSocket traffic and from mitmproxy, client, and server Docker logs. Only ciphertext-bearing message frames were visible to the intercepting proxy, with no plaintext leakage observed.                                                                                                     |
| Check browser console output for accidental plaintext leakage                          | PASS | browser developer console captured after sending a unique encrypted test token, showing no plaintext token or related sensitive message content                                              | With E2E enabled, the unique test token did not appear in the browser console. No accidental plaintext leakage was observed in console output.                                                                                                                                                                                                            |
| Check error paths for accidental plaintext or sensitive internal-detail leakage        | PASS | Malformed sequence error path was reproduced with the standalone harness, and browser, server, and mitmproxy outputs were reviewed for plaintext and sensitive internal details.             | Malformed input was rejected and no plaintext leakage was observed in the reviewed server or mitmproxy logs. SECURITY FINDING: One browser-visible 4001 error exposed Python-specific exception text (int() can't convert non-string with explicit base), revealing unnecessary internal implementation detail.                                           |
| Summarize where plaintext is expected, observed, unexpectedly exposed, or absent       | PASS | Review of the E2E-off and E2E-on mitmproxy captures, Docker log searches, browser console check, and malformed error-path reproduction.                                                      | Plaintext was expected and observed in mitmproxy with E2E disabled. With E2E enabled, the plaintext token was absent from mitmproxy traffic, container logs, and the browser console. SECURITY FINDING: One error path exposed Python-specific exception text, revealing unnecessary internal implementation detail despite no plaintext message leakage. |

| Session 7: Architecture, exposure, and residual visibility                                                     |      |                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Attempt direct host access to localhost:3000                                                                   | PASS | Direct curl attempt to http://localhost:3000 showing connection refused on both ::1 and 127.0.0.1.                                                                                                                     | Direct host access to the client service on localhost:3000 was blocked because the container port is not published to the host. The client remained reachable only through the intended proxied architecture.                                                                                                                                                                                                            |
| Attempt direct host access to localhost:8000                                                                   | PASS | Direct curl attempt to http://localhost:8000 showing connection refused on both IPv6 and IPv4.                                                                                                                         | Direct host access to the backend server on localhost:8000 was blocked because the container port is not published to the host. The backend remained inaccessible through direct host access.                                                                                                                                                                                                                            |
| Attempt direct access to the backend WebSocket                                                                 | PASS | Direct WebSocket connection attempt to ws://localhost:8000/ws showing ConnectionRefusedError because the backend port was not published to the host.                                                                   | Direct host access to the backend WebSocket was blocked. The connection attempt to ws://localhost:8000/ws was refused, confirming that the backend WebSocket was not directly reachable from the host.                                                                                                                                                                                                                   |
| Confirm mitmproxy is the only published ingress in the deployed Docker architecture                            | PASS | Docker container port mapping showing only mitmproxy with host-published ports, while the client and server expose internal container ports only.                                                                      | Mitmproxy was confirmed as the only published ingress in the deployed Docker architecture. The client on port 3000 and backend server on port 8000 were not directly published to the host.                                                                                                                                                                                                                              |
| Record metadata still visible with HPKE on: timing, message count, ciphertext size, and connection information | PASS | Mitmproxy WebSocket and connection captures showing encrypted message frames while still exposing message count, exact timestamps, ciphertext and overall flow size information, and client/server connection details. | RESIDUAL RISK: With HPKE enabled, plaintext message content was not visible to mitmproxy. Observable traffic metadata remained, including four WebSocket messages, exact message timestamps, ciphertext and overall flow size information, SNI, TLS version and cipher, and client/server connection addresses. This is an expected limitation of payload encryption rather than an implementation failure.              |
| Attempt to force an E2E downgrade                                                                              | PASS | Browser capture showing the tampered server_hello caused the encrypted channel to be torn down, plus mitmproxy logs confirming the server_hello enc value was modified in transit.                                     | An active intermediary modified the server_hello handshake material. The browser detected the fault, tore down the encrypted channel, and did not silently downgrade to plaintext.                                                                                                                                                                                                                                       |
| Attempt to tamper with browser-side encryption code before the prompt is encrypted                             | PASS | Mitmproxy logs showing successful injection of client-side plaintext-capture code and recovery of the unique plaintext token before HPKE encryption.                                                                   | SECURITY FINDING: The TLS-terminating intermediary successfully modified browser-delivered code and captured a unique plaintext token before HPKE encryption, while the browser still appeared healthy and the E2E channel remained established. HPKE itself was not broken; the result demonstrates that end-to-end confidentiality depends on the integrity of the browser-side encryption code and its delivery path. |
| Session 8: Final review and findings                                                                           |      |                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Review every completed test and confirm its final status                                                       | DONE | Completed review of all tests from Sessions 1–7, including final statuses, evidence references, and notes for expected behavior, security findings, implementation bugs, and residual risks.                           | All completed tests were reviewed and assigned a final status. No previously completed test remained unresolved. Findings and implementation issues were preserved in the notes for later classification.                                                                                                                                                                                                                |
| Rerun only unexpected, failed, or unclear results                                                              | DONE | Reviewed all completed tests for unexpected, failed, or unclear outcomes and confirmed that previously questionable results had already been reproduced or clarified during Sessions 2, 5, and 6.                      | No additional reruns were necessary. Unexpected behaviors had already been independently reproduced or further tested, including the non-canonical server pin encoding, wrong-length sequence handling, malformed ciphertext errors, and browser-visible                                                                                                                                                                 |

|                                                                                        |      |                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|----------------------------------------------------------------------------------------|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                        |      |                                                                                                                                                                              | Python-specific exception text.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Separate implementation bugs from genuine security findings                            | DONE | Classification labels added to the relevant completed test notes.                                                                                                            | Observed results were classified as genuine security findings, implementation bugs, residual risks, expected behavior, or successful security controls.                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Select the strongest 3–5 findings and clearest exhibits for the paper and presentation | DONE | Reviewed all findings from Sessions 1, 2, 5, 6, and 7 and consolidated duplicate reproduction and summary rows into five unique issues with associated screenshots and logs. | Five unique red-team findings/issues were identified: acceptance of a non-canonical Base64URL encoding of the pinned Ed25519 identity key; inconsistent error handling for malformed sequence values, including a generic internal error for wrong-length input; inconsistent error handling for malformed ciphertext encodings; browser-visible exposure of Python-specific exception text; and successful client-code tampering by the TLS-terminating intermediary that captured plaintext before HPKE encryption while the browser still appeared healthy and the E2E channel remained established. |